

CS5239 Performance Analysis of Computer Systems

Lecture 1 - Introduction

L01: Introduction

- Motivation and definition of performance analysis
- Tradeoffs
- Metrics
- Observability
- Steps in performance analysis

A running case: checkout latency doubled

A small service has changed. Users report slower checkout, but the cause is unclear.

BEFORE

210 ms

NOW

420 ms

CPU

42%

ERRORS

0.8%

What do we know?

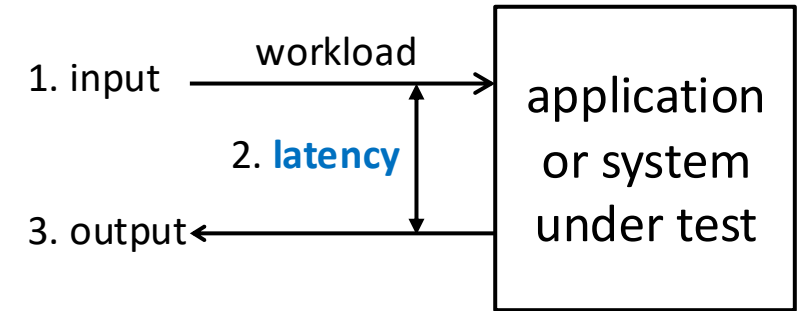
The symptom is real: latency increased.

What do we not know?

Which component, workload, or interaction caused it?

What is one plausible hypothesis and one measurement to take?

Performance Definition



- Performance describes how well a system completes a job under a workload
 - Latency - time per operation
 - Throughput - operations per unit time
 - Utilization - fraction of capacity used
 - Saturation - work delayed because capacity is full
- Always qualify the metric: average latency? read throughput? per core?
- Performance is important to achieve:
 - Time deadline
 - Efficient utilization of computing resources
 - Good usage of energy

Performance of a computer system

- Performance of a computer system is multidimensional
 - complex component interactions
 - hard to predict how it will scale
 - sharing of resources leads to congestion/queueing

Latency tells you where time went

A 100 ms database query is not one indivisible 100 ms event.

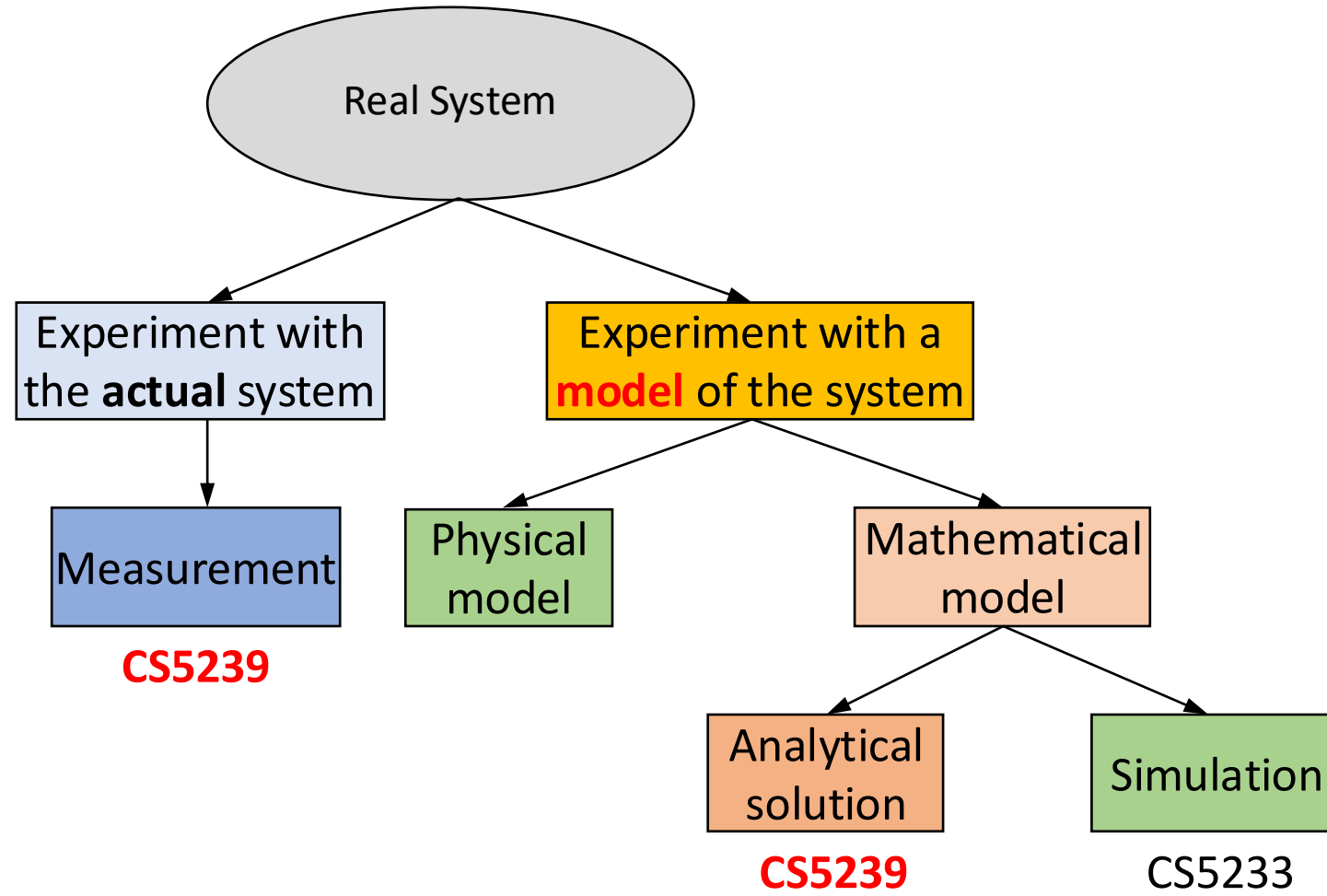


If the disk wait is eliminated, the best possible speedup is:

$$100 / 20 = 5x$$

Interpretation: decomposition turns a symptom into a prioritized next action.

Ways to Study a System



Three ways to study a system

Use the least expensive approach that can answer the question with enough confidence.

Measure

Run the real system

+ High credibility; real interactions

– Only one configuration; can perturb the system

Model

Represent relationships mathematically

+ Fast insight; exposes limits and trade-offs

– Assumptions may not hold

Simulate

Program the important behaviour

+ Explore many scenarios cheaply

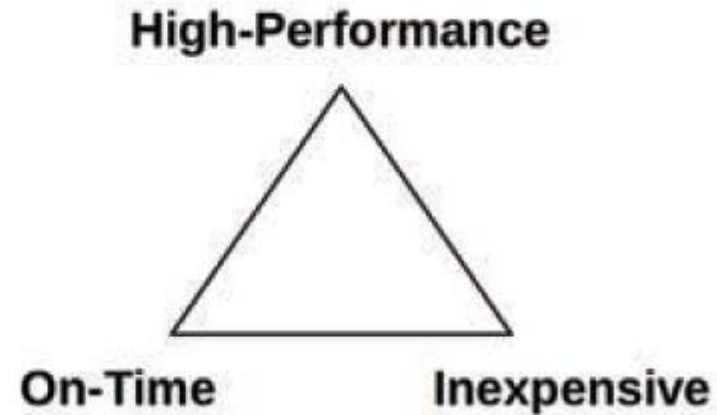
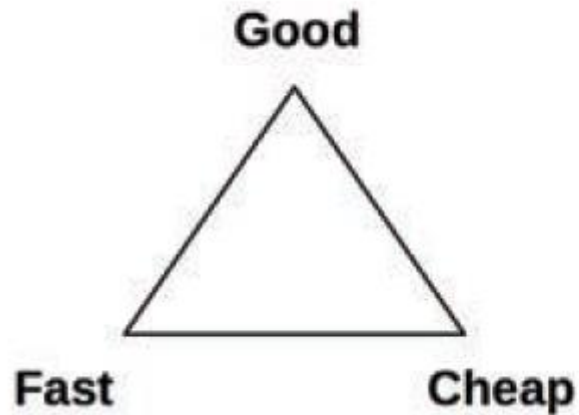
– Model detail and validation take work

Performance Evaluation

- Performance evaluation is of interest to multiple stakeholders:
 - Users
 - System administrators
 - Developers
 - Designers
- Goal: obtain or provide **highest performance** at the **lowest cost**
performance-cost ratio

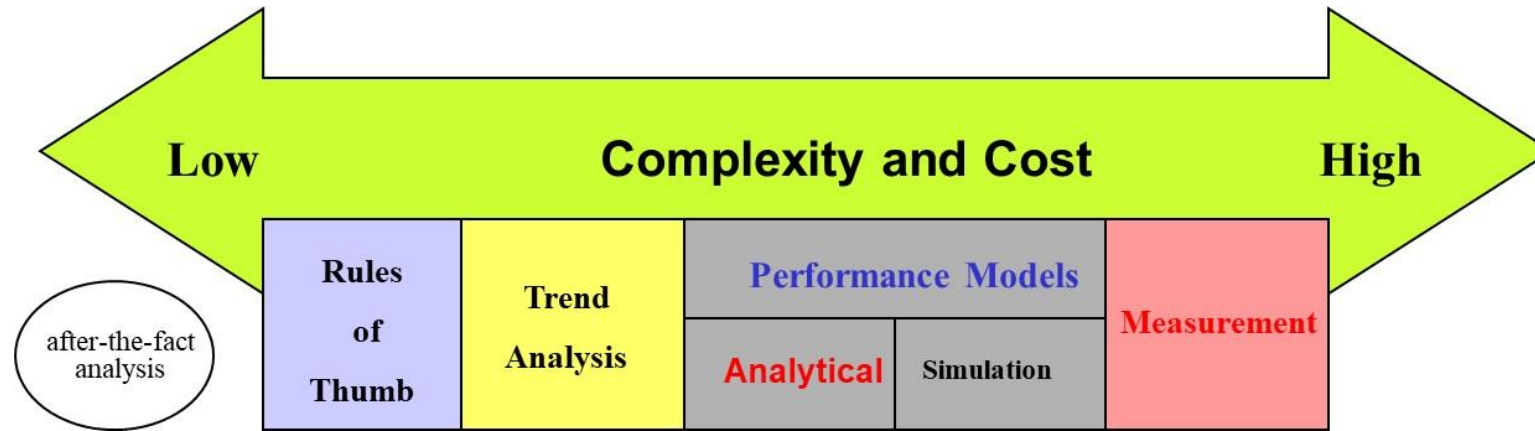
Trade-offs

- The good/fast/cheap “pick two” trade-off



- Many IT projects choose on-time and inexpensive, leaving performance to be fixed later

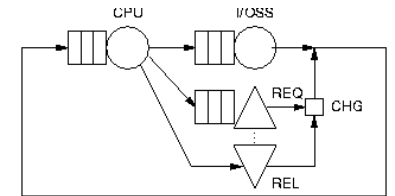
Complexity and cost



- Measurements of actual systems
- Modeling
 - Simulation using software models
 - Analytical modeling using techniques such as queueing analysis

Performance Evaluation Techniques (1)

- Analytical modelling
 - Mathematical representation of the system
 - Model assumptions may not be met in practice - lower accuracy
 - Ease of changing parameters
 - Useful for insight and validating measurements/simulations
- Simulation
 - Program written to model important features of the system
 - Can be easily modified to study impact of changes
 - Cost – writing and running the program
 - Impossible to model every small detail – sometimes low accuracy

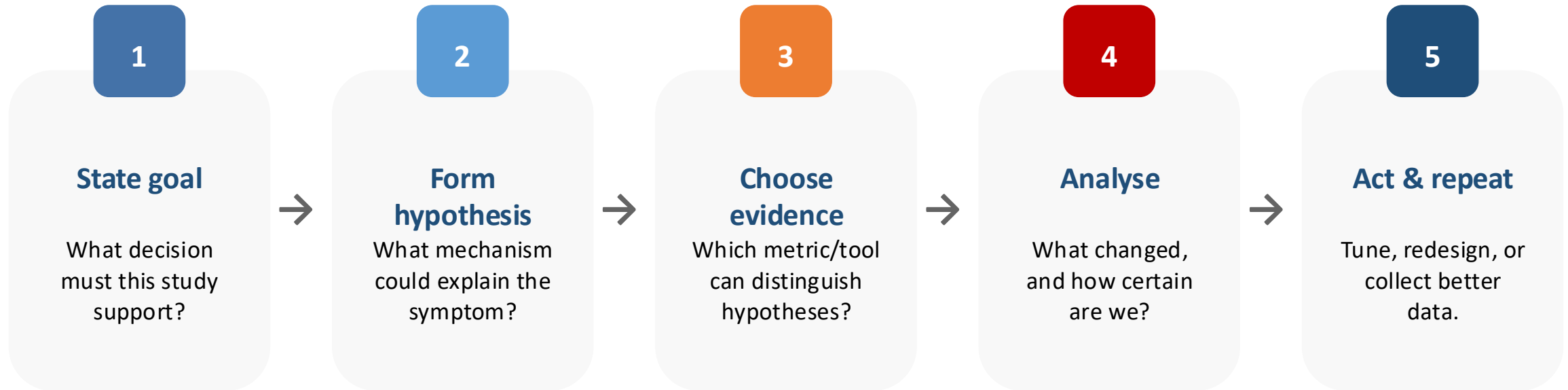


Models are abstraction of the system

Performance Evaluation Techniques (2)

- Measurement
 - No simplifying assumptions
 - Highest credibility
 - Only works if systems / applications exist
 - Information only on specific system being measured
 - Harder to change system parameters in a real system
 - May be intrusive and perturb operation of systems
 - Difficult and time consuming
 - Need for software tools

A Practical Workflow: Question → Evidence → Decision



This workflow is the spine of conducting performance analysis

Goals of Performance Analysis

- Set expectations → service level
 - Provide information for users
- System tuning & performance debugging
 - Find parameters that produce best overall performance
 - Identify bottlenecks
 - Determine impact of a new feature
- Compare alternative designs
 - Which configurations are best under which conditions
 - Which program/algorithm is faster
- ...

When to Conduct Performance Analysis?

- Selection/configuration of computer systems:
 - System upgrade
 - System design: design of applications and equipment
- Analysis of existing systems: **capacity planning** predicts when future load levels will saturate the system and determines cost-effective way of delaying system saturation

Performance Analysis as Part of the System's Design

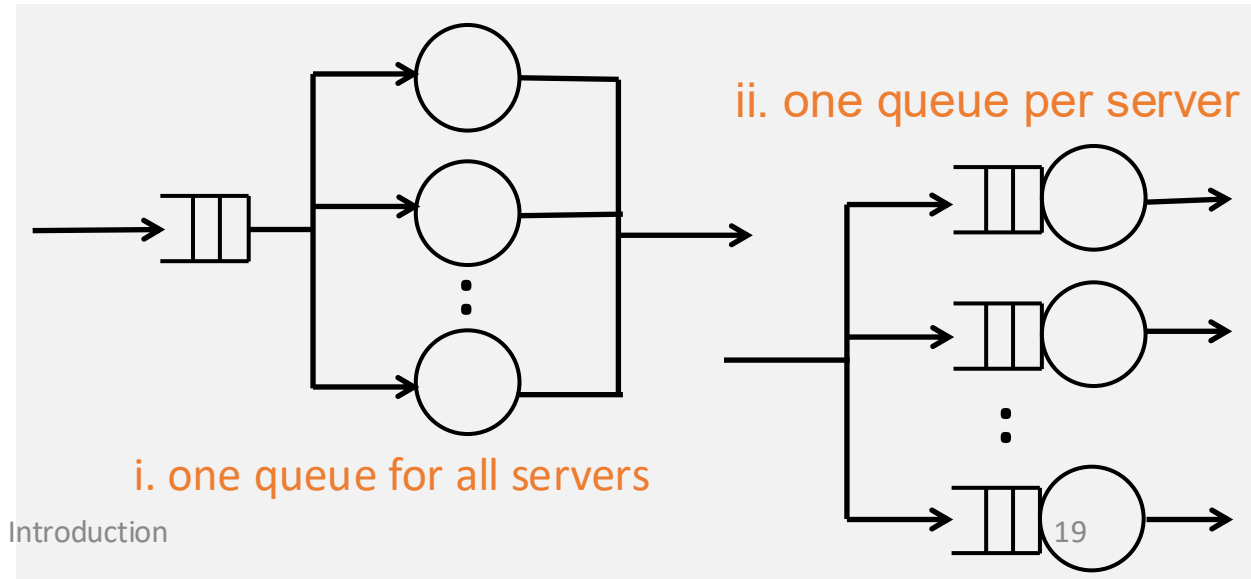
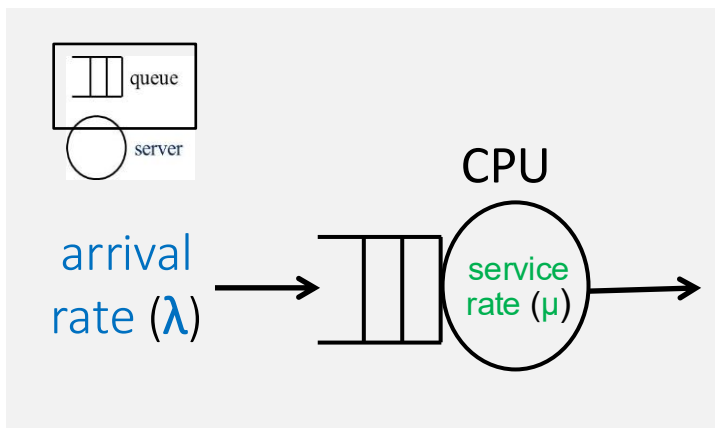
- Questions to ask
 - What deadline must the system meet?
 - What resources and energy may it consume?
 - What is the cost of improving the target metric?
 - Which users or workloads benefit?
- Points of failure in the performance analysis process
 - Optimize after a problem appears
 - Choose a configuration using one metric
 - Ignore resource sharing and congestion
 - Treat performance as someone else's responsibility

Performance Activities

- Should **follow the life cycle** of a software project from conception through development to production deployment
- Set objectives and create a **performance model** as a first step in product development
 - Don't defer performance engineering work to a later time, after a problem arises
- **Capacity planning** – study the resource footprint of development software to see how well the design can meet the target needs
- Cloud applications performance
 - *Canary testing*
 - *Blue-green development*

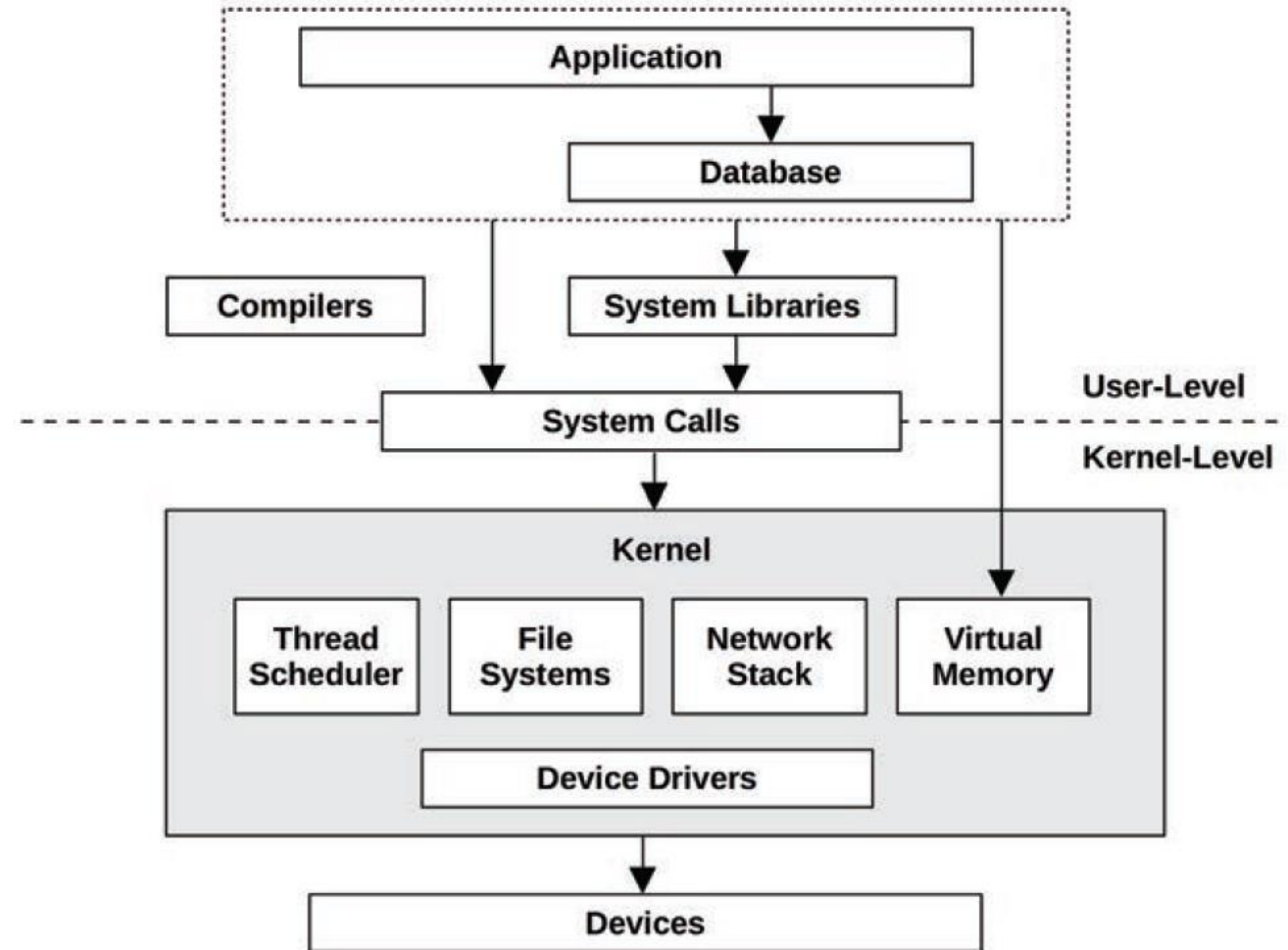
Performance Questions can be Framed as Queueing Questions

1. What is the **average time** it takes a job to complete service? [**latency**]
2. What is the **throughput** of the system (number of jobs completed per unit time)? [**bandwidth**]
3. If **arrival rate is doubled** ($\lambda \rightarrow 2\lambda$), do we do nothing or how much should the **service rate** (μ) increase?
4. If we need more server (CPU) capacity, what are our options?
 - a. Buy a **new server** with the needed capacity
 - b. Buy a **few smaller servers** that adds up to the required capacity
 - c. How do we organize these servers?
 - i. One queue for all servers
 - ii. One queue for each server
 - iii. Does it matter?



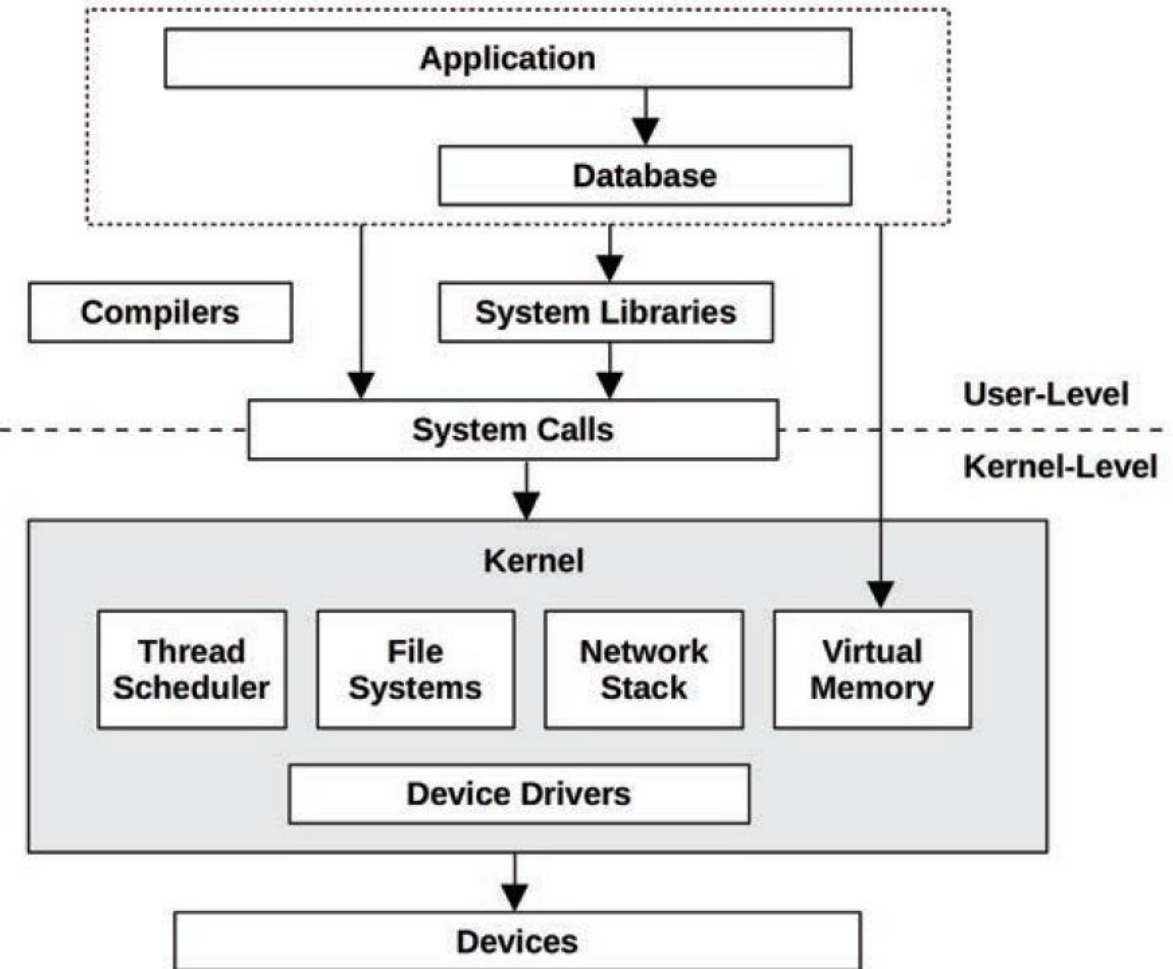
Systems Performance

- Study the performance of an entire computer system
 - All major software and hardware components
 - Anything in the data path, from storage devices to application software
 - For distributed systems, multiple servers and applications



Systems Performance

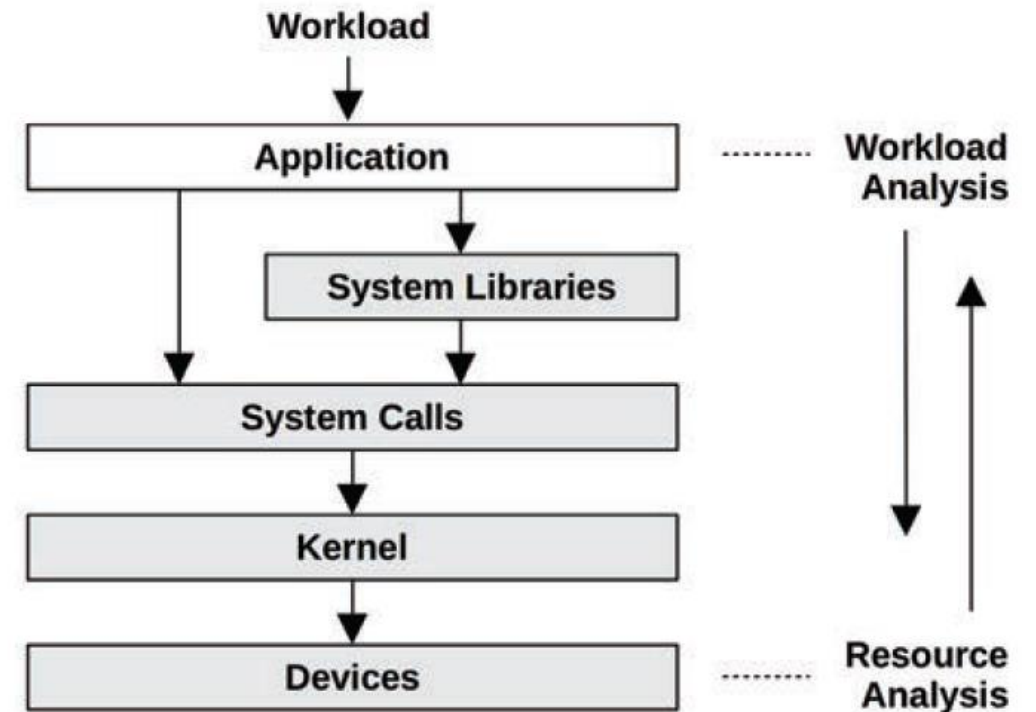
- Study the performance of the full system stack
- **Full stack** → the entire software stack from the application down to metal (the hardware)
 - Includes system libraries, the kernel, and the hardware
 - Includes the compiler



Perspectives

- The resource analysis perspective
→ system administrators
- The workload analysis perspective
→ application developers

Operating
System
Software
Stack



Challenges

- Subjectivity
 - “The average disk I/O response time is 1 ms.” → is it good or bad?
- Complexity
 - Lack of an obvious starting point for analysis
 - Start with a hypothesis
 - Cascading failure – one failed component causes performance issues in others
 - Bottlenecks – complex and unexpectedly related to each other
- Multiple causes: issues do not have a single root cause
- Multiple performance issues
 - Quantify the magnitude of issues

Known and Unknowns

US Secretary of Defence Donald Rumsfeld, 2002

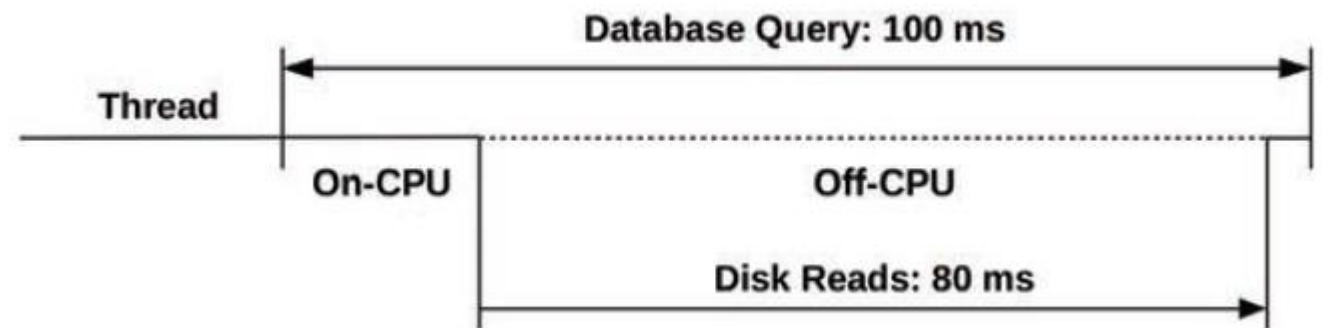
- There are **known knowns**; there are things we know we know
 - Example: you know that you should check the CPU utilization and you know utilization is 80%
- We also know there are **known unknowns**; that is to say we know there are some things we do not know.
 - Example: you know that you could be checking what is making the CPU busy but have yet to do so
- But there are also **unknown unknowns** – there are things we do not know we don't know.
 - Example: you may not know what device interrupts are making the CPU busy and thus you are not checking them
- Performance issues can originate from anywhere, including areas of the system that you know nothing about and therefore not checking (unknown unknowns)

Metrics

- Metrics are needed to **quantify** the performance of a system
- Example: **Latency** - measure of time spent waiting
 - Broadly, it means the time for any operation to complete
 - Time taken by an application request, a database query, a file system operation, etc.
 - Website: click to screen paint
 - Database: query submission to result
 - Network: connection setup, request, or full transfer
 - Use average, median, tail percentile, and window where relevant

Latency needs a qualifier

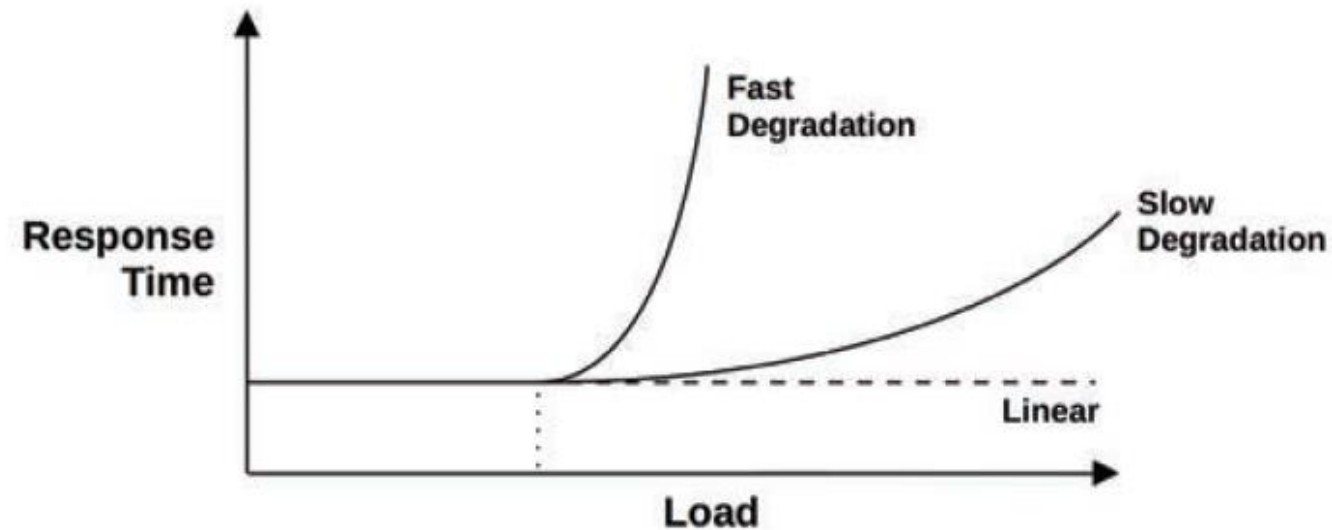
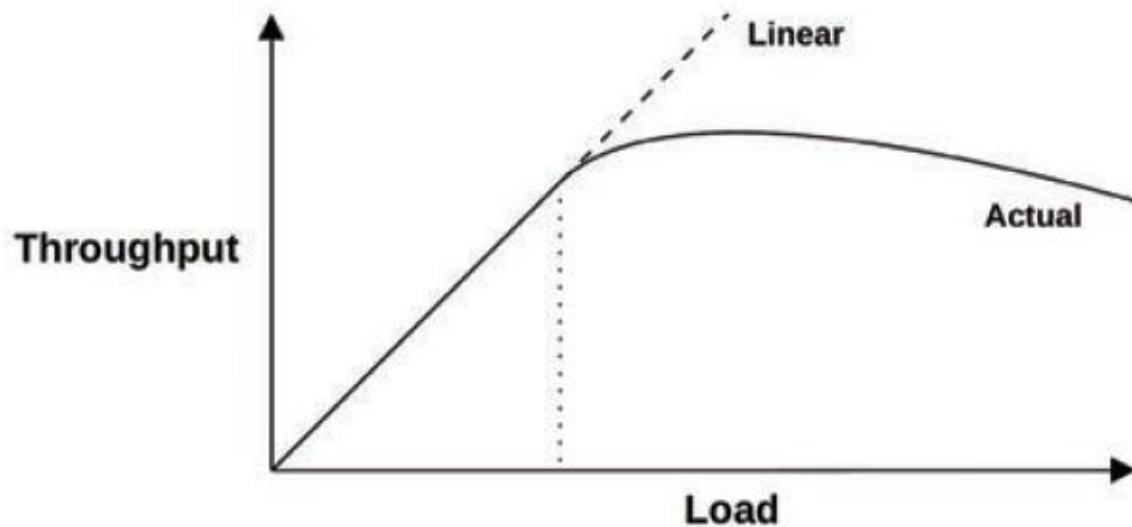
- A database query that takes 100 ms [**latency**] during which it spends 80 ms blocked waiting for disk reads
 - Decompose the time
- The maximum performance improvement is obtained by eliminating disk reads (e.g., by caching) from 100 ms to 20 ms
 - 5x faster [**speedup**]



- What if you are looking at I/O operations per second instead?

Scalability

- The performance of the system under increasing load
 - Linear scalability until the **knee point**
 - A component reaches 100% utilization: **saturation point**
 - Queueing is frequent



Scalability and Saturation

Before the knee point

- Throughput may scale approximately linearly
- Resources have spare capacity
- Small changes can still be hidden by averages
- Queueing may be present but not yet dominant

At the knee point and beyond

- A resource approaches full utilisation
- Waiting time grows rapidly
- Queues form and latency rises
- Adding capacity or changing demand becomes necessary

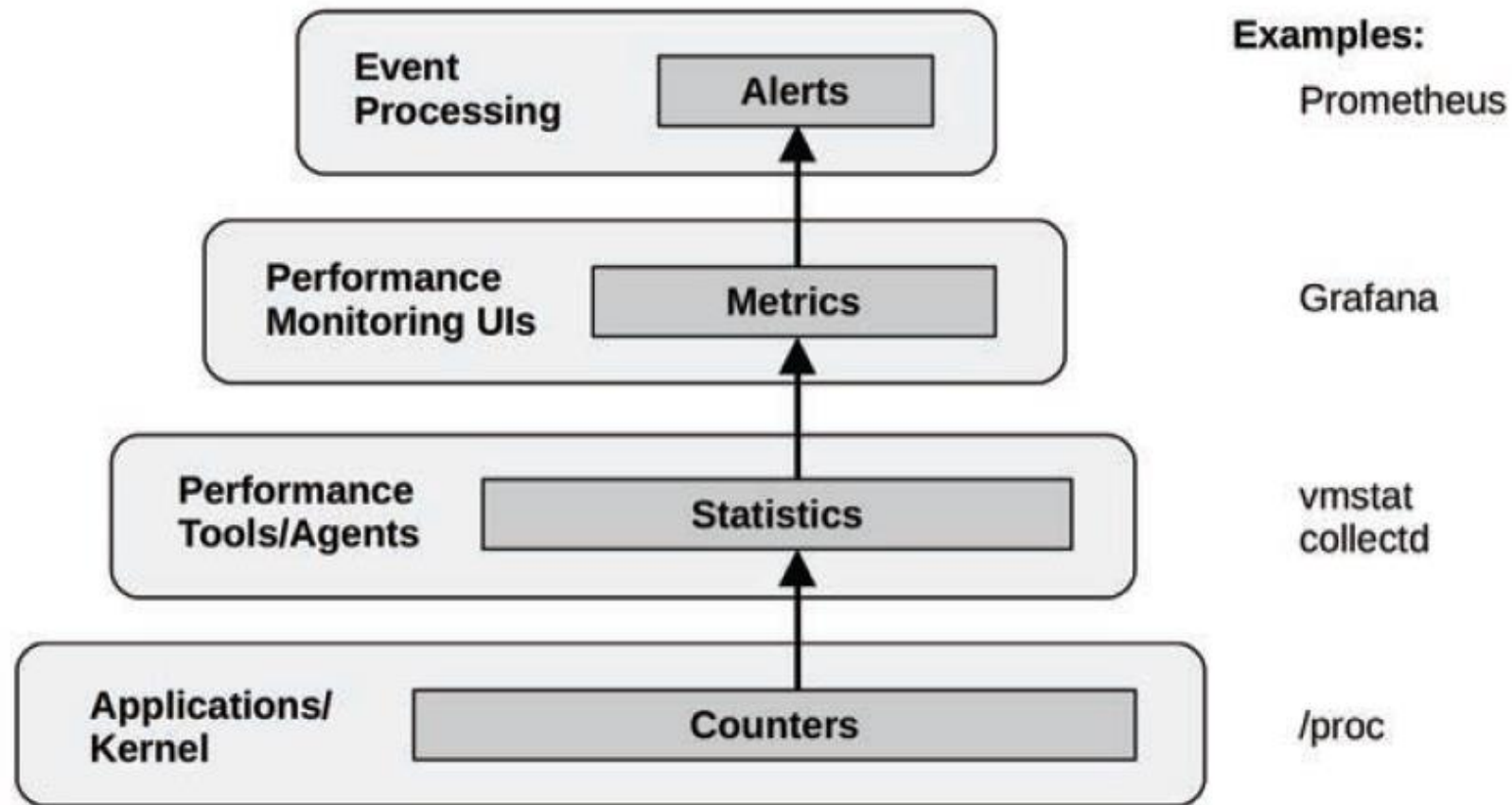
Observability

- Understanding a system through observation using different observability tools
- Observability tools use
 - Counters
 - Profiling
 - Tracing
- For production environments, observability tools may perturb production workloads through resource contention

Counters, Statistics, Metrics, and Alerts

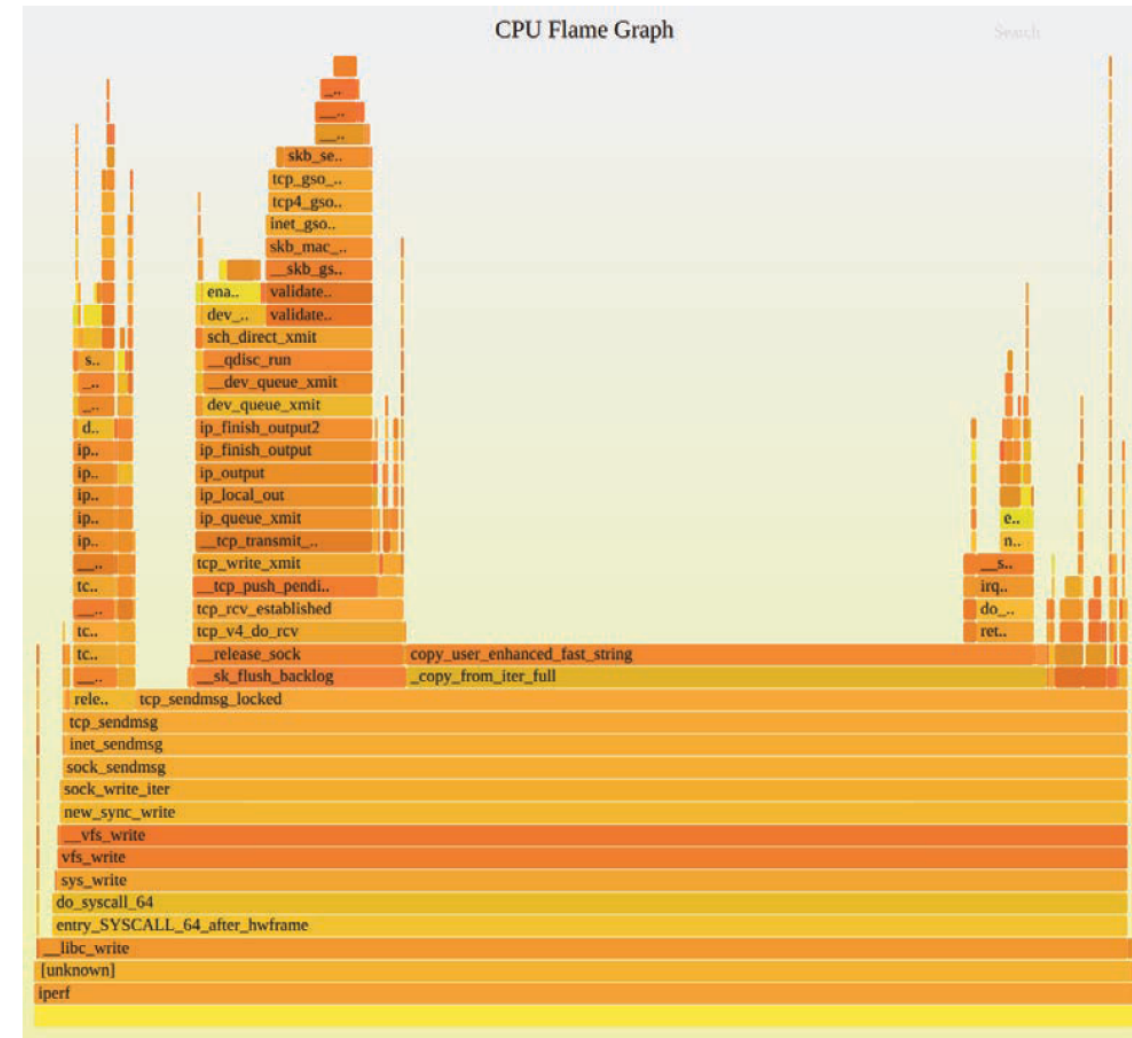
- Applications and the kernel typically provide data on their state and activity:
 - Example: operation counts byte counts, latency measurements, resource utilization, and error rates
 - Implemented as integer variables called **counters**
 - Hard-coded in the software
 - Some are cumulative, and they always increment
- Cumulative counters can be read at different times by performance tools for calculating **statistics**
 - Examples: the rate of change over time, the average, percentiles, etc.
 - **Metric** is a statistic that has been selected to evaluate or monitor a target
 - Monitoring software can also support creating custom **alerts** from these metrics

Performance Instrumentation Terminology



Profiling

- Refers to the use of tools that perform sampling: taking a subset (a sample) of measurements to paint a coarse picture of the target
 - CPUs are a common profiling target
 - Take timed-interval samples of the on-CPU code paths
- Visualizing the profiles is important
 - Example: CPU flame graphs

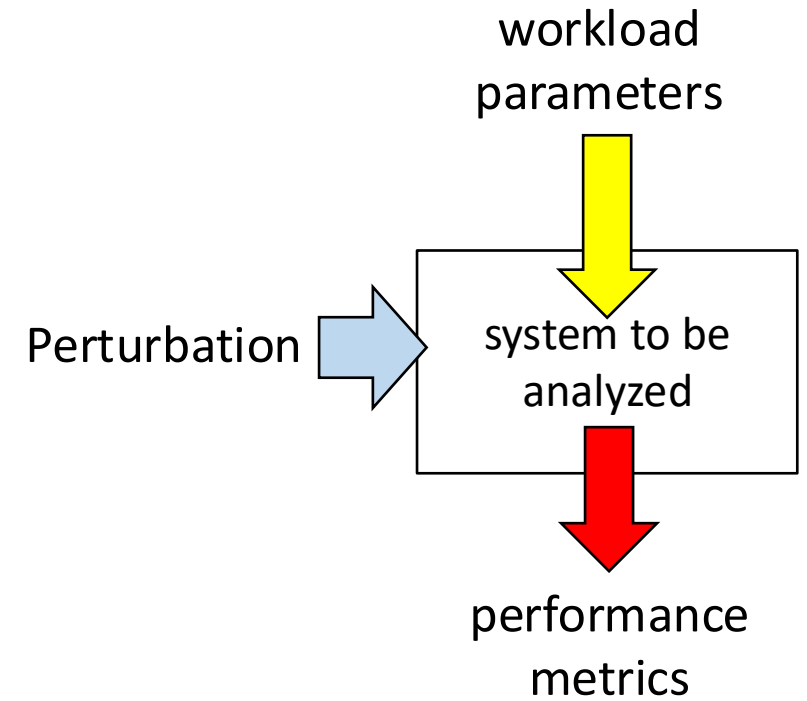


Tracing

- Event-based recording: event data is captured and saved for later analysis or consumed on-the-fly for custom summaries
 - Special-purpose tracing tools for system calls (Linux strace(1)) and network packets (e.g., Linux tcpdump(8))
 - General-purpose tracing tools that can analyze the execution of all software and hardware events (e.g., Linux Ftrace, BCC, and bpftrace).
- Static instrumentation: hard-coded software instrumentation points added to the source code
- Dynamic instrumentation creates instrumentation points after the software is running, by modifying in-memory instructions to insert instrumentation routines

Steps in Performance Evaluation Study

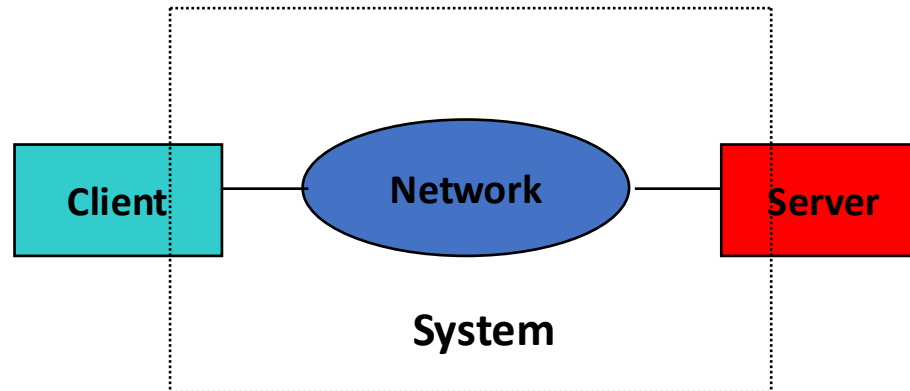
1. State the goals of the study and define the system **boundaries**
2. List system **services** and possible **outcomes**
3. Select performance **metrics**
4. List system and workload **parameters**
5. Select **factors** and their values
6. Select evaluation **techniques**
7. Select the **workload**
8. Design the **experiments**
9. Analyze and interpret the **data**
10. Present the **results**. Start over, if necessary



Example: Performance Evaluation Study

1. System definition:

- compares performance of applications using remote pipes (caller is not blocked) with remote procedure calls (calling program is blocked)



Example (2)

2. Service:

- two types of channel calls: remote procedure call and remote pipe

3. Metrics:

- resources are local computer (client), remote computer (server), and network link
- performance metrics:
 - a. Elapsed time per call
 - b. Maximum call rate per unit of time, or equivalently, the time required to complete a block of n successive calls
 - c. Local CPU time per call
 - d. Remote CPU time per call
 - e. Number of bytes sent on the link per call

Example (3)

4. Parameters

- i. workload parameters (resource demand)
 - a. Time between successive calls
 - b. Number and sizes of the call parameters and results
 - c. Types of channel
 - d. Other loads on the local and remote CPUs, network
- ii. System parameters (resource capacity)
 - a. Speed of the local and remote CPUs
 - b. Speed of the network
 - c. Operating system overhead for interfacing with the channels and the network
 - d. Reliability of the network affecting the number of retransmissions required

Example (4)

5. Key Factors:

- a. Types of channel: 2 types – compares remote pipes and remote procedure calls
- b. Speed of the network: 2 remote host locations - short distance (in the campus) and long distance (across the country)
- c. Sizes of call parameters to be transferred: small and large
- d. Number n of consecutive calls: 11 values of n – 1, 2, 4, 8, 16, 32,..., 512, 1024

Factors are selected based on resource availability and objectives of evaluation

Example (5)

6. Evaluation technique

- Measurements are used for evaluation
- Analytical modeling is used to justify the consistency of measured values for different parameters

7. Workload

- Synthetic program generating the specified types of channel requests
- Monitor resources consumed and log the measured results

8. Experimental design

- Full factorial experimental design consists of 88 (23 x 11) experiments

9. Data Analysis

- Variance to quantify the effects of the first three factors
- Regression to quantify the effects of the number n of successive calls

10. Data Presentation

- Final results plotted as a function of the block size n

Summary

Performance analysis turns a vague symptom into evidence-based action.

1. Define performance precisely

Latency, throughput, utilization, and saturation describe different aspects of a system and must be qualified by workload and measurement context.

2. Study the whole system

Performance can be affected by the application, workload, operating system, hardware, storage, network, and interactions between components.

3. Choose the right evaluation method

Measurement, simulation, and analytical modelling are complementary. Each makes different assumptions and supports different decisions.

4. Start with a hypothesis

State the goal, identify workload and factors, select metrics, and collect evidence that can distinguish competing explanations.

References

- [Gregg] – chapter 1
- [Jain] – chapters 2 & 3